

Universidad Nacional Autonoma de Mexico

Facultad de Ingeniería



Modelos de Programación Orientado a Objetos

Número de Practica:9

Integrantes del equipo:Leon Acosta Diego y Leon Carrasco Jonatan Abdel

Fecha de entrega:30/10/26

Grupo:6

1. Evidencia de ejecución

- Salida de Main.java

```
jonca@Jonatan MINGW64 ~/JonatanCarrasco/mpoo-2027-1/entregas/leon_jonatan/p09 (entregas_leon_jonatan)
$ java -cp out mazmorra.Main
Al empezar:
Conan (Caballero) 120/120
Merlin (Mago) 80/80
Legolas (Arquero) 100/100
Despues de una ronda:
Conan (Caballero) 110/120
Merlin (Mago) 62/80
Legolas (Arquero) 75/100
Espada: durabilidad 99
```

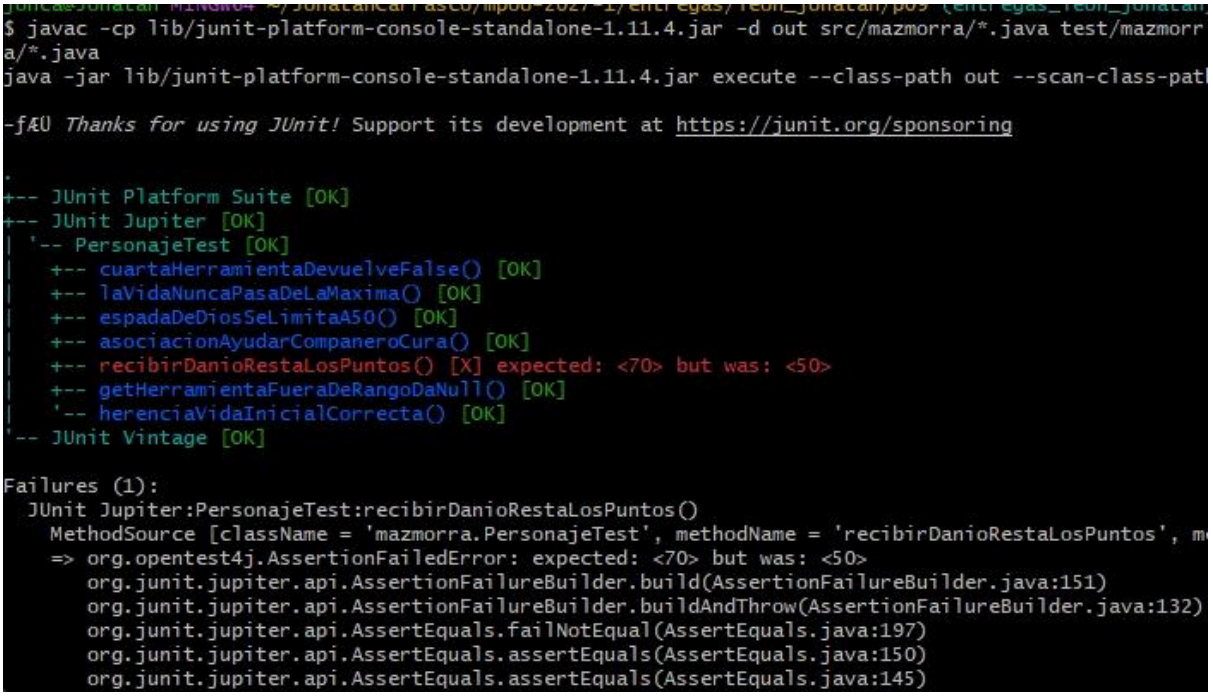
- Salida de TrampasDeAcceso.java

```
$ javac -cp out -d out src/trampas/TrampasDeAcceso.java
src\trampas\TrampasDeAcceso.java:10: error: vida has private access in Personaje
        heroe.vida = 9999;
              ^
src\trampas\TrampasDeAcceso.java:11: error: inventario has private access in Personaje
        heroe.inventario = new Inventario();
              ^
src\trampas\TrampasDeAcceso.java:14: error: durabilidad has private access in Herramienta
        espada.durabilidad = 1000000;
               ^
src\trampas\TrampasDeAcceso.java:17: error: herramientas has private access in Inventario
        bolsa.herramientas[0] = espada;
              ^
src\trampas\TrampasDeAcceso.java:18: error: cuantas has private access in Inventario
        bolsa.cuantas = 50;
              ^
5 errors
```

- Salida de Tramposo.java

```
$ javac -cp out -d out src/trampas/Tramposo.java
java -cp out trampas.Tramposo
BLOQUEADA  1) golpe negativo para curarse
BLOQUEADA  2) curarse arriba del tope de vida
BLOQUEADA  3) forjar una espada de 5000 de poder
BLOQUEADA  4) cargar una cuarta herramienta
BLOQUEADA  5) pedir la herramienta 99
BLOQUEADA  6) atacar sin ninguna herramienta
BLOQUEADA  7) revivir a un personaje muerto
```

- Pruebas unitarias con JUnit



2. Cuestionario y Respuestas analiticas

1. Las cuatro relaciones (Herencia, Composición, Agregación y Asociación):

- Herencia (extends): Se observa en la declaración de las clases hijas, por ejemplo: public class Caballero extends Personaje. Al ejecutar el constructor mediante super(nombre, 120);, se heredan y reutilizan los atributos y métodos del padre. ¿Qué le pasa a la parte si el dueño desaparece?.

  R= Al ser una extensión estructural la clase hija depende totalmente de la existencia de la clase base.

- Composición: Se ve reflejada en la creación interna del inventario dentro del constructor de la clase Personaje mediante this.inventario = new Inventario();. ¿Qué le pasa a la parte si el dueño desaparece?

  R= Muere con el dueño ya que como el inventario se fabrica internamente dentro del objeto mediante un new, su ciclo de vida está totalmente acoplado al personaje.

- Agregación: Se manifiesta cuando un personaje equipa herramientas externas a través de public boolean equipar(Herramienta h) { return inventario.agregar(h); }. ¿Qué le pasa a la parte si el dueño desaparece?:

R=Sobrevive sin el dueño. La herramienta fue instanciada de forma independiente fuera del personaje, por lo que si el personaje o el inventario se destruyen entonces la herramienta continúa existiendo en la memoria.

- Asociación: Se establece mediante la conexión de compañeros con el método public void setCompanero(Personaje companero) { this.companero = companero; }. ¿Qué le pasa a la parte si el dueño desaparece?:

  R= Pueden sobrevivir de forma independiente, al ser un enlace o referencia debil entre 2 objetos que colaboran entre si, si uno de los personajes muere o desaparece, el otro no se ve afectado estructuralmente.

2. Los huecos de seguridad, encapsulamiento y diseño de métodos:

- ¿Alguna trampa funcionó la primera vez? ¿Qué cambiaste?

  R= No, ninguna funciono la primera vez que se probó, gracias a que se siguieron estrictamente las reglas de diseño de diagrama de clases y se aplicación las validaciones lógicas , el codigo rechazó todas las trampas de manera predefinida.

  Tomando la trampa 3, esta es bloqueada directamente en la clase Herramienta por la línea: this.danio = entre (danio, 1, 50). Dicha instrucción convoca a un método privado que evalúa los límites mediante Math.max y Math.min forzando así a que cualquier valor exagerado se reduzca automáticamente al limite permitido (50).

- ¿Por qué el diagrama NO tiene setVida ni getHerramientas?:

  R=Con la ausencia de SetVida se protege el encapsulamiento para evitar que cualquier jugador tramposo modifique los puntos de vida de manera arbitraria como por ejemplo si asigna un valor terriblemente absurdo como 9999, toda modificación debe pasar por metodos de negocios seguros y controlados que validan las reglas de los puntos y topes de vida máxima. En el caso de getHerramientas el retornar la referencia al arreglo interno (Herramienta[]) rompería la seguridad de la estructura, permitiendo que código externo modifique los elementos o agregue más herramientas de las permitidas sin pasar por la validación de la capacidad del método.

3. Análisis de la prueba roja

- Copia el mensaje y explica qué significa cada número y quién estaba mal:

  Mensaje: org.opentest4j.AssertionFailedError: expected: <70> but was: <50>

  Explicación:

  R=El valor expected: <70> representa el resultado que la prueba unitaria afirmaba de manera errónea mientras que was: <50> es el valor numérico real que se calculó en la ejecución.

¿Quién estaba mal?

R= El código fuente estaba bien y la prueba unitaria contiene una aserción incorrecta escrita así por la guia de la práctica, esto ocurre ya que la prueba creo un personaje de tipo Mago con una vida máxima de 80 y se le aplicó un daño de 30 puntos de vida, esto nos da como resultado 50 lo que nos dice que la matematica del programa operó de manera correcta, pero la prueba esperaba 70 demostrando que la falla vino de una expectativa que era erronea y no de un fallo de lógica.